

Hyper parameter tunning

video [🔗](#)

github [🔗](#)

Note Random Search CV -> chose randomly from all the parameter , instend of trying all the possible combination like gride search cv

value of scoring

For classification metrics

- Accuracy: scoring='accuracy'
- Precision: scoring='precision'
- Recall: scoring='recall'
- F1 Score: scoring='f1'
- Mean Absolute Error (MAE): scoring='neg_mean_absolute_error'
- Mean Squared Error (MSE): scoring='neg_mean_squared_error'
- R² Score: scoring='r2'
- Root Mean Squared Error (RMSE): Need to create custom scoring function for it
- Adjusted R² Score: Need to create custom scoring function for it

For Regression metrics

code for custom Root Mean Squared Error (RMSE) and Adjusted R² Score:

```
from sklearn.metrics import make_scorer, r2_score, mean_squared_error
import numpy as np

# Custom RMSE scorer
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

# Custom Adjusted R^2 scorer
def adjusted_r2(y_true, y_pred, n, p):
    r2 = r2_score(y_true, y_pred)
    adj_r2 = 1 - (1 - r2) * ((n - 1) / (n - p - 1))
    return adj_r2

# Create custom scorers
rmse_scorer = make_scorer(rmse, greater_is_better=False) # We want to minimize rmse, hence greater_is_better=False
adj_r2_scorer = make_scorer(adjusted_r2, greater_is_better=True) # We want to maximize adjusted r2, hence greater_is_better=True

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression

# Define model
model = LinearRegression()

# Parameter grid
param_grid = {
    'fit_intercept': [True, False],
    'normalize': [True, False]
}

# GridSearchCV with custom scoring
grid_search_rmse = GridSearchCV(model, param_grid, cv=5, scoring=rmse_scorer)
grid_search_adj_r2 = GridSearchCV(model, param_grid, cv=5, scoring=adj_r2_scorer)

# Fit GridSearchCV
grid_search_rmse.fit(X_train, y_train)
grid_search_adj_r2.fit(X_train, y_train)

# Best RMSE
best_rmse = -grid_search_rmse.best_score_ # negate to get positive value
print(f"Best RMSE: {best_rmse:.4f}")

# Best Adjusted R^2
best_adj_r2 = grid_search_adj_r2.best_score_
print(f"Best Adjusted R^2: {best_adj_r2:.4f}")
```

code

GridSearchCV

```
# Number of trees in random forest
n_estimators = [20, 60, 100, 120]

# Number of features to consider at every split
max_features = [0.2, 0.6, 1.0]

# Maximum number of levels in tree
max_depth = [2, 8, None]

# Number of samples
max_samples = [0.5, 0.75, 1.0]

param_grid = {'n_estimators': n_estimators,
              'max_features': max_features,
              'max_depth': max_depth,
              'max_samples': max_samples
             }

rf = RandomForestClassifier()

from sklearn.model_selection import GridSearchCV

rf_grid = GridSearchCV(estimator=rf,
                      param_grid=param_grid,
                      cv=5,
                      scoring='accuracy',
                      verbose=2,
                      n_jobs=-1)

rf_grid.fit(X_train, y_train)

rf_grid.best_params_

rf_grid.best_score_
```

RandomSearchCV

```
# Number of trees in random forest
n_estimators = [20, 60, 100, 120]

# Number of features to consider at every split
max_features = [0.2, 0.6, 1.0]

# Maximum number of levels in tree
max_depth = [2, 8, None]

# Number of samples
max_samples = [0.5, 0.75, 1.0]

# Bootstrap samples
bootstrap = [True, False]

# Minimum number of samples required to split a node
min_samples_split = [2, 5]

# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2]

param_grid = {'n_estimators': n_estimators,
              'max_features': max_features,
              'max_depth': max_depth,
              'max_samples': max_samples,
              'bootstrap': bootstrap,
              'min_samples_split': min_samples_split,
              'min_samples_leaf': min_samples_leaf
             }

from sklearn.model_selection import RandomizedSearchCV

rf_grid = RandomizedSearchCV(estimator=rf,
                             param_distributions=param_grid,
                             cv=5,
                             scoring='accuracy',
                             verbose=2,
                             n_jobs=-1)

rf_grid.fit(X_train, y_train)

rf_grid.best_params_

rf_grid.best_score_
```

Optuna

Here we can even treat ml algo as a hyper parameter

Here we use Bayesian optimization to do smart hyper parameter tuning

study

Task of doing hyperparameter training and finding the best parameter for your data

trying different-different combination of values for parameter is called trial

1 trial = 1 combination of parameter values

study = combination of multiple trial

trial parameters

For a trial, Parameter has a value which we called trial parameter value

objective function

It take paramater value and return the accuracy/r2 valeu (which we want to increase)

sampler

who decide which value we should use next

how to implement

First define a objective function

create a study

!pip install optuna

import optuna

```
def objective(trial):
    # Suggest values for the hyperparameters
    n_estimators = trial.suggest_int('n_estimators', 50, 200)
    max_depth = trial.suggest_int('max_depth', 3, 20)

    # Create the RandomForestClassifier with suggested hyperparameters
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42
    )

    # Perform 3-fold cross-validation and calculate accuracy
    score = cross_val_score(model, X_train, y_train, cv=3, scoring='accuracy').mean()

    return score # Return the accuracy score for Optuna to maximize
```

here 50,200 -> mean -> all value between 50 to 200 in n_estimators

```
# Create a study object and optimize the objective function
study = optuna.create_study(direction='maximize', sampler=optuna.samplers.TPESampler()) # We aim to maximize accuracy
study.optimize(objective, n_trials=50) # Run 50 trials to find the best hyperparameters
```

```
# Print the best result
print("Best trial accuracy: {study.best_trial.value}")
print("Best hyperparameters: {study.best_trial.params}")
```

```
best_model = RandomForestClassifier(**study.best_trial.params, random_state=42)
```

A single algo

```
from optuna.visualization import plot_optimization_history, plot_parallel_coordinate,
plot_slice, plot_contour, plot_param_importances

Optimization History      plot_optimization_history(study).show()
Parallel Coordinates Plot  plot_parallel_coordinate(study).show()
Slice Plot                plot_slice(study).show()
Contour Plot              plot_contour(study).show()
Hyperparameter Importance plot_param_importances(study).show()
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
```

```
# Define the objective function for Optuna
def objective(trial):
    # Choose the algorithm to tune
    classifier_name = trial.suggest_categorical('classifier', ['SVM', 'RandomForest', 'GradientBoosting'])

    if classifier_name == 'SVM':
        # SVM hyperparameters
        c = trial.suggest_float('C', 0.1, 100, log=True)
        kernel = trial.suggest_categorical('kernel', ['linear', 'rbf', 'poly', 'sigmoid'])
        gamma = trial.suggest_categorical('gamma', ['scale', 'auto'])

        model = SVC(C=c, kernel=kernel, gamma=gamma, random_state=42)

    elif classifier_name == 'RandomForest':
        # Random Forest hyperparameters
        n_estimators = trial.suggest_int('n_estimators', 50, 300)
        max_depth = trial.suggest_int('max_depth', 3, 20)
        min_samples_split = trial.suggest_int('min_samples_split', 2, 10)
        min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 10)
        bootstrap = trial.suggest_categorical('bootstrap', [True, False])

        model = RandomForestClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            min_samples_split=min_samples_split,
            min_samples_leaf=min_samples_leaf,
            bootstrap=bootstrap,
            random_state=42
        )

    elif classifier_name == 'GradientBoosting':
        # Gradient Boosting hyperparameters
        n_estimators = trial.suggest_int('n_estimators', 50, 300)
        learning_rate = trial.suggest_float('learning_rate', 0.01, 0.3, log=True)
        max_depth = trial.suggest_int('max_depth', 3, 20)
        min_samples_split = trial.suggest_int('min_samples_split', 2, 10)
        min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 10)

        model = GradientBoostingClassifier(
            n_estimators=n_estimators,
            learning_rate=learning_rate,
            max_depth=max_depth,
            min_samples_split=min_samples_split,
            min_samples_leaf=min_samples_leaf,
            random_state=42
        )

    # Perform cross-validation and return the mean accuracy
    score = cross_val_score(model, X_train, y_train, cv=3, scoring='accuracy').mean()

    return score
```

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
```

```
study.trials_dataframe()
```

```
To know how much time each algo run = (to know which algo you need to focus)
```

```
study.trials_dataframe()[params[classifier].value_counts()
```

other imp things

```
study.trials_dataframe().groupby('params[classifier']["value"].mean()
```